02

DEEP DIVE 02 · దాదాపు ప్రతి పెద్ద కంపెనీ అడిగేది

Design an LRU & LFU Cache

> Google · Amazon · Microsoft · Meta · Apple — ఈ question యొక్క ఏదో ఒక రూపం అన్ని చోట్లా వస్తుంది. LeetCode lo ఇది ఒక puzzle; interview lo ఇది ఒక **design** ప్రశ్న.

ఇక్కడ మనం మూడు versions రాస్తాం, మరియు మొదటి రెండూ **విరిగిపోతాయి** – ఒకటి తప్పు జవాబు ఇచ్చి, ఇంకొకటి నెమ్మదిగా ఉండి. ఆ నెమ్మదితనాన్ని **కొలిచి చూపిస్తాను**: 183 ms నుంచి 1 ms కి.

ఈ doc ఎలా చదవాలి

పక్కన editor తెరిచి కలిసి రాయండి. ఇక్కడి **ప్రతి సంఖ్యా నిజంగా కొలిచినదే** – timing numbers కూడా. మీ machine lo కొంచెం వేరే సంఖ్యలు రావొచ్చు, కానీ **నిష్పత్తి** అదే ఉండాలి. అదే ముఖ్యం.

> **ఈ PROBLEM గురించి ఒక అపోహ**
>
> చాలా మంది దీన్ని ఒక **LeetCode puzzle** అనుకుంటారు – "HashMap + doubly linked list, అంతే". ఆ జవాబు సరైనదే, కానీ interview lo అది **సగం మాత్రమే**.
>
> అసలు ప్రశ్నలు ఇవి: **ఎందుకు** doubly linked list? Singly సరిపోదా? LRU కాకుండా LFU కావాలంటే ఎంత code మారాలి? TTL ఎక్కడ పెడతారు? Thread safety? – ఇవి design ప్రశ్నలు, మరియు ఈ doc వాటి గురించే.

విషయ సూచిక (Table of Contents)

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. Cache అంటే ఏమిటి – మరియు అసలు కష్టం ఎక్కడ
2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

Part 2 – మొదటి version, మరియు మొదటి విరుపు

3. Step – హద్దు ఉన్న ఒక cache
4. **మొదటి విరుపు** – అత్యంత వేడి key నే తీసేశాం

Part 3 – రెండో విరుపు: సరైన జవాబు, కానీ నెమ్మది

5. Step – వాడకం క్రమాన్ని గుర్తుపెట్టుకోవడం
6. **రెండో విరుపు** – నెమ్మదితనాన్ని కొలిచి చూడటం
7. `indexOf` + `splice` ఎందుకు ఖరీదు

Part 4 – O(1) కి చేరడం

8. Step – HashMap + Doubly Linked List
9. Doubly ఎందుకు, singly ఎందుకు సరిపోదు
10. కొలత – 183 ms నుంచి 1 ms కి

Part 5 – LFU: పూర్తిగా వేరే ప్రశ్న

11. LRU vs LFU – ఏది ఎప్పుడు తప్పు జవాబు ఇస్తుంది
12. **మూడో విరుపు** – naive LFU మరియు దాని O(n) scan
13. Step – Frequency buckets తో O(1) LFU

Part 6 – ఒకే Cache, పలు policies

14. Step – Eviction ని బయట పెట్టడం
15. Step – TTL, మరియు "గడియారం ఎవరిది"
16. మొత్తం code ఒకే చోట · నడిపి చూద్దాం

Part 7 – Interview lo

17. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి
18. నోటితో చెప్పాల్సిన English script
19. Follow-ups – thread safety, distributed cache
20. ఏమి నేర్చుకున్నాం

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

---

1. Cache అంటే ఏమిటి – మరియు అసలు కష్టం ఎక్కడ

Cache = **ఖరీదైన పనిని మళ్ళీ చేయకుండా, ఫలితాన్ని దాచుకోవడం.** DB query 50 ms తీసుకుంటే, ఆ ఫలితాన్ని memory lo పెట్టి తర్వాతిసారి 0.05 ms lo ఇవ్వడం.

ఇది సులభం. కష్టం ఎక్కడంటే – **memory పరిమితం.**

10 లక్షల rows ఉన్న table ని మొత్తం memory lo పెట్టలేం. కాబట్టి cache కి ఒక **capacity** ఉంటుంది. అది నిండిపోయాక, కొత్తది చేర్చాలంటే **ఏదో ఒకటి తీసేయాలి**.

> **ఇదే మొత్తం problem:** *ఏది తీసేయాలి?*

ఈ ఒక్క ప్రశ్నకి జవాబే `LRU`, `LFU`, `FIFO`, `Random`, `TTL` – ఈ పేర్లన్నీ. అవి వేర్వేరు data structures కాదు; **అవి ఒకే ప్రశ్నకి వేర్వేరు జవాబులు.**

CACHE యొక్క రెండు భాగాలు · ఒకటి సులభం, ఒకటి కష్టం

ఒక cache lo నిజంగా రెండు వేర్వేరు సమస్యలు ఉన్నాయి

1 · నిల్వ చేయడం (సులభం)
key → value
get, put – రెండూ O(1)
ఒక HashMap చాలు
ఇక్కడ ఆలోచించాల్సినది ఏమీ లేదు

2 · ఎవరిని తీసేయాలి (కష్టం)
నిండినప్పుడు ఒక్కరిని ఎంచుకోవాలి
ఆ ఎంపిక కూడా O(1) కావాలి
మరియు ఆ **నియమం మారుతుంది**
ఈ doc మొత్తం దీని గురించే

"ఏది తీసేయాలి" – ప్రసిద్ధ జవాబులు

FIFO – మొదట వచ్చినది · సులభం, కానీ వాడకాన్ని పట్టించుకోదు (§4 lo ఇది విరుగుతుంది)
LRU – చాలాసేపటి నుంచి ఎవరూ చదవనిది · ఆచరణలో అత్యుత్తమం, అందుకే ఇదే default
LFU – అతి తక్కువసార్లు చదివినది · స్థిరమైన traffic కి మేలు, కానీ ఒక ఉచ్చు ఉంది (§11)
TTL – ఒక నిర్ణీత కాలం తర్వాత పోవాలి · పైవాటికి **అదనంగా**, ప్రత్యామ్నాయంగా కాదు

2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

ప్రశ్న	జవాబు నా design ని ఎలా మారుస్తుంది
Eviction policy ఏమిటి – LRU నా?	LRU అయితే DLL; LFU అయితే buckets (§13). **ముందే అడగాలి**
`get` కూడా "వాడకం" గా లెక్కా?	అవును అంటే `get` lo కూడా order మార్చాలి – ఇది చాలా మంది మర్చిపోతారు
ఉన్న key కి `put` చేస్తే?	విలువ మారాలి, మరియు అది ఒక వాడకంగా లెక్కకావాలి
Capacity 0 అయితే?	ఒక edge case – `put` ఏమీ చేయకూడదు
TTL కావాలా?	ఉంటే ప్రతి entry కి `expiresAt`, మరియు "గడువు తీరినది hit కాదు" (§15)
Thread-safe కావాలా?	ఇది **మరో లేవనెత్తాలి** – §19
`get` దొరకకపోతే ఏమి తిరిగి ఇవ్వాలి?	`-1` (LeetCode), `null`, `undefined`, లేక exception – ఇది ఒక API నిర్ణయం

ఆ చివరి ప్రశ్న ఒక ఉచ్చు

LeetCode lo `-1` తిరిగి ఇస్తారు. కానీ నిజమైన cache lo ఒక key యొక్క విలువ **నిజంగానే** `-1` కావొచ్చు! అప్పుడు "దొరకలేదు" మరియు "విలువ −1" మధ్య తేడా తెలియదు.

ఇది interview lo చెప్పదగిన మంచి పరిశీలన: *"నేను `undefined` తిరిగి ఇస్తాను, లేదా ఒక `{ found, value }` jodi. `-1` అనేది LeetCode యొక్క సరళీకరణ – అది నిజమైన API lo ఒక bug."*

Part 2 – మొదటి version, మరియు మొదటి విరుపు

3. Step – హద్దు ఉన్న ఒక cache

అతి సులభమైన cache: ఒక Map, ఒక capacity. నిండితే **మొదట చేర్చినదాన్ని** తీసేయడం (FIFO).

```
class FifoCache {
  constructor(capacity) { this.capacity = capacity; this.map = new Map(); }

  get(key) { return this.map.has(key) ? this.map.get(key) : -1; }

  put(key, value) {
    if (this.map.size >= this.capacity && !this.map.has(key)) {
      const oldest = this.map.keys().next().value;   // JS Map చేర్చిన క్రమం గుర్తుంచుకుంటుంది
      this.map.delete(oldest);
      console.log(`   తీసేశాం: ${oldest}`);
    }
    this.map.set(key, value);
  }
}
```

ఇది ఏం చేస్తుంది: నిండేవరకు చేరుస్తుంది; నిండాక, అతి పాత entry ని తీసేసి కొత్తది పెడుతుంది.

`this.map.keys().next().value` అనేది JavaScript `Map` యొక్క ఒక లక్షణాన్ని వాడుతోంది – **అది keys ని చేర్చిన క్రమంలో ఉంచుతుంది**. ఇది ఇక్కడ ఉపయోగపడింది, మరియు §8 lo మళ్ళీ వస్తుంది.

4. మొదటి విరుపు – అత్యంత వేడి key నే తీసేశాం

ఇప్పుడు ఒక నిజమైన వాడకపు నమూనా పెడదాం. మూడు keys చేర్చి, అందులో `A` **ని ఐదుసార్లు** చదువుదాం – అంటే అది అత్యంత ఉపయోగపడే key. ఆపై ఒక కొత్తది చేరుద్దాం.

```
const c = new FifoCache(3);
c.put('A', 1); c.put('B', 2); c.put('C', 3);

console.log('A ని 5 సార్లు చదివాం:', [1,2,3,4,5].map(() => c.get('A')).join(','));
console.log('ఇప్పుడు D చేరుస్తున్నాం…');
c.put('D', 4);
console.log('A ఇంకా ఉందా?', c.get('A'));
console.log('cache lo ఉన్నవి:', [...c.map.keys()].join(', '));
```

```
A ని 5 సార్లు చదివాం: 1,1,1,1,1
ఇప్పుడు D చేరుస్తున్నాం…
   తీసేశాం: A
A ఇంకా ఉందా? -1
cache lo ఉన్నవి: B, C, D
```

మొదటి విరుపు

ఐదుసార్లు చదివిన `A` ని తీసేశాం. ఒక్కసారి కూడా ఎవరూ ముట్టుకోని `B`, `C` బతికిపోయాయి.

ఇది ఒక సాంకేతిక bug కాదు – code సరిగ్గానే పనిచేసింది. ఇది ఒక **విధానపరమైన తప్పు**. FIFO ఒక్కటే విషయం చూస్తుంది: ఎప్పుడు వచ్చావు? కానీ cache కి నిజంగా ముఖ్యమైనది: ఎప్పుడు పనికొచ్చావు?

తర్వాతిసారి `A` కావాలంటే మళ్ళీ DB కి వెళ్ళాలి. అంటే మనం cache పెట్టిన **ఉద్దేశమే** దెబ్బతింది.

సరైన ప్రశ్న

FIFO **చేర్చిన** క్రమం చూస్తోంది. మనకి కావలసినది **చదివిన** క్రమం.

అంటే: *"చాలాసేపటి నుంచి ఎవరూ చదవని key ఏది?"* – దాన్నే **Least Recently Used (LRU)** అంటారు.

ఈ ఒక్క మార్పు – "చేర్చిన" నుంచి "చదివిన" కి – మొత్తం data structure ని మారుస్తుంది. ఎందుకంటే **చదవడం కూడా ఇప్పుడు క్రమాన్ని మార్చాలి.**

Part 3 – రెండో విరుపు: సరైన జవాబు, కానీ నెమ్మది

5. Step – వాడకం క్రమాన్ని గుర్తుపెట్టుకోవడం

సహజమైన ఆలోచన: **keys ని ఒక array lo వాడకపు క్రమంలో ఉంచడం.** చదివినప్పుడల్లా ఆ key ని చివరికి తీసుకెళ్ళడం (చివర = అతి కొత్తది). తీసేయాల్సి వస్తే – మొదటిది.

```
class ArrayLru {
  constructor(capacity) { this.capacity = capacity; this.map = new Map(); this.order
= []; }

  #touch(key) {
    const i = this.order.indexOf(key);      // ← ఈ key ఎక్కడ ఉంది?
    if (i !== -1) this.order.splice(i, 1);  // ← అక్కడి నుంచి తీసేయడం
    this.order.push(key);                   // చివర = అతి కొత్తది
  }

  get(key) {
    if (!this.map.has(key)) return -1;
    this.#touch(key);                       // చదవడం కూడా ఒక వాడకమే
    return this.map.get(key);
  }

  put(key, value) {
    if (!this.map.has(key) && this.map.size >= this.capacity) {
      const lru = this.order.shift();       // మొదటిది = అతి పాతది
      this.map.delete(lru);
    }
    this.map.set(key, value);
    this.#touch(key);
  }
}
```

మొదటి విరుపు పరీక్షని మళ్ళీ నడుపుదాం – ఈసారి `A` ని చదివి, ఆపై `D` చేరుద్దాం:

```
cache lo: A, C, D | A బతికిందా? true
```

`A` **బతికింది**, `B` పోయింది. **మొదటి విరుపు సరిచేయబడింది** – ఇప్పుడు cache నిజంగా వాడకాన్ని గౌరవిస్తోంది.

ఇది సరైన జవాబు. LeetCode lo ఇది pass అవుతుంది (చిన్న inputs కి). కాబట్టి **ఇక్కడ ఆగిపోవచ్చా?**

6. రెండో విరుపు – నెమ్మదితనాన్ని కొలిచి చూడటం

"ఇది నెమ్మది" అని చెప్పడం సులభం. **కొలిచి చూపిద్దాం.** వేర్వేరు capacities తో, 20,000 reads:

```
for (const n of [1000, 10000, 50000]) {
  const c = new ArrayLru(n);
  for (let i = 0; i < n; i++) c.put(i, i);          // నింపడం

  const t = process.hrtime.bigint();
  for (let i = 0; i < 20000; i++) c.get(i % n);     // 20,000 reads
  const ms = Number(process.hrtime.bigint() - t) / 1e6;
  console.log(`capacity ${String(n).padStart(6)} → 20,000 reads: ${ms.toFixed(0)}
ms`);
}
```

```
capacity   1000 → 20,000 reads: 4 ms
capacity  10000 → 20,000 reads: 17 ms
capacity  50000 → 20,000 reads: 183 ms
```

ఈ సంఖ్యల గురించి ఒక నిజాయితీ మాట. ఇవి ఒక నిజమైన run నుంచి తీసినవే, కానీ timing ప్రతిసారీ కొంచెం మారుతుంది – నేను ఐదుసార్లు నడిపితే ఆ చివరి వరుస **116 నుంచి 185 ms** మధ్య ఊగింది (JIT warm-up, మిగతా processes). మొదటి రెండు వరుసలు మాత్రం స్థిరంగా ఉన్నాయి.

కాబట్టి **ఖచ్చితమైన సంఖ్యని కాదు, వాలుని చూడండి** – capacity పెరిగితే సమయం పెరుగుతోంది. అదే ఇక్కడ ముఖ్యం, మరియు అది ప్రతి run lo నిజం.

రెండో విరుపు – ఈ సంఖ్యలని జాగ్రత్తగా చూడండి

Capacity **50 రెట్లు** పెరిగింది (1,000 → 50,000). సమయం **46 రెట్లు** పెరిగింది (4 ms → 183 ms).

అంటే ప్రతి `get` యొక్క ఖర్చు cache పరిమాణంతో **పాటు పెరుగుతోంది**. ఇది సరిగ్గా **cache అనే ఆలోచనకి వ్యతిరేకం** – cache పెద్దదైతే ఎక్కువ hits రావాలి, కానీ ఇక్కడ పెద్దదైతే నెమ్మదవుతోంది.

10 లక్షల entries ఉన్న ఒక నిజమైన cache lo, ప్రతి read ఒక DB call కంటే నెమ్మదవుతుంది. అప్పుడు cache ఒక **సహాయం కాదు, ఒక భారం**.

---

7. `indexOf` + `splice` ఎందుకు ఖరీదు

`#touch` lo మూడు పంక్తులు ఉన్నాయి. వాటి ఖర్చు చూద్దాం:

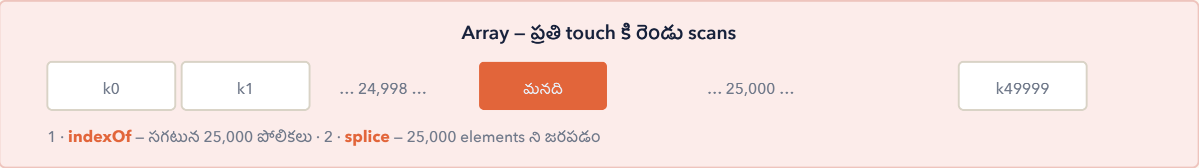
పంక్తి	ఏం చేస్తుంది	ఖర్చు
`this.order.indexOf(key)`	మొదటి నుంచి వెతుకుతుంది, key దొరికేవరకు	**O(n)**
`this.order.splice(i, 1)`	ఆ తర్వాతి అన్ని elements ని ఒక స్థానం ఎడమకి జరుపుతుంది	**O(n)**

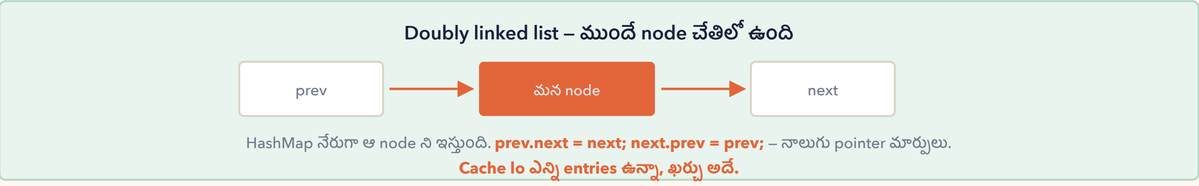
పంక్తి	ఏం చేస్తుంది	ఖర్చు
`this.order.push(key)`	చివర చేరుస్తుంది	O(1)

రెండు O(n) operations, ప్రతి `get` కీ. అదే ఆ 183 ms.

ARRAY LO ఒక KEY ని ముందుకి తేవడం · ఎంత పని

50,000 entries · మధ్యలో ఉన్న ఒక key ని చివరికి తేవాలి





అవసరమైనవి రెండు

Array విఫలమవుతోంది ఎందుకంటే దానికి రెండూ ఒకేసారి రావట్లేదు:

1. **"ఈ key ఎక్కడ ఉంది?" – O(1) lo తెలియాలి.** → ఇది HashMap ఇస్తుంది.
2. **"దాన్ని ఇక్కడి నుంచి తీసి అక్కడ పెట్టు" – O(1) lo కావాలి.** → ఇది linked list ఇస్తుంది.

కాబట్టి జవాబు: **రెండిటినీ కలపడం.** HashMap `key → node`; node ఒక doubly linked list lo ఉంటుంది.

Part 4 — O(1) కి చేరడం

8. Step — HashMap + Doubly Linked List

```
class Node {
  constructor(key, value) { Object.assign(this, { key, value, prev: null, next: null
}); }
}

class LruCache {
  #map = new Map();                      // key → node
  #head = new Node(null, null);          // sentinel: జాబితా ప్రారంభం
  #tail = new Node(null, null);          // sentinel: జాబితా ముగింపు

  constructor(capacity) {
    this.capacity = capacity;
    this.#head.next = this.#tail;        // ఖాళీ జాబితా: head ↔ tail
    this.#tail.prev = this.#head;
  }

  #remove(n)   { n.prev.next = n.next; n.next.prev = n.prev; }
  #addFront(n) { n.next = this.#head.next; n.prev = this.#head;
                 this.#head.next.prev = n; this.#head.next = n; }

  get(key) {
    const n = this.#map.get(key);
    if (!n) return -1;
    this.#remove(n); this.#addFront(n);   // చదివాం → ముందుకి
    return n.value;
  }

  put(key, value) {
    const existing = this.#map.get(key);
    if (existing) {
      existing.value = value;
      this.#remove(existing); this.#addFront(existing);
      return;
    }
    if (this.#map.size >= this.capacity) {
      const lru = this.#tail.prev;        // చివరిది = అతి పాతది
      this.#remove(lru); this.#map.delete(lru.key);
    }
    const n = new Node(key, value);
    this.#map.set(key, n); this.#addFront(n);
  }
}
```

Sentinel nodes ఎందుకు

`#head` మరియు `#tail` నిజమైన data కాదు – అవి **ఖాళీ కాపలా nodes**. ఇవి లేకపోతే `#remove` ఇలా ఉండేది:

```
#remove(n) {
  if (n.prev) n.prev.next = n.next; else this.first = n.next;     // మొదటి node నా?
  if (n.next) n.next.prev = n.prev; else this.last = n.prev;      // చివరి node నా?
}
```

నాలుగు `if` లు, మరియు ప్రతిదీ ఒక bug అవకాశం. Sentinels తో **ఏ node కైనా `prev` మరియు `next` ఎప్పుడూ ఉంటాయి** – కాబట్టి ఒక్క `if` కూడా అవసరం లేదు.

> **Interview lo ఇది చెప్పండి:** *"I'm using sentinel head and tail nodes so that no node is ever at a boundary. That removes every null check from remove and insert, which is where linked-list bugs normally come from."* – ఇది ఒక చిన్న వివరం, కానీ ఇది మీరు linked lists ని ఇంతకుముందు రాశారని చూపిస్తుంది.

నడిపి చూద్దాం:

```
మొదలు (కొత్తది ముందు): C → B → A
A చదివాక:           A → C → B
D చేర్చాక:          D → A → C | B పోయిందా? true
```

ప్రతి అడుగూ సరిగ్గా ఉంది: `A` చదివాక అది ముందుకి వచ్చింది, `B` చివరికి జారింది, మరియు `D` వచ్చినప్పుడు `B` పోయింది.

---

9. Doubly ఎందుకు, singly ఎందుకు సరిపోదు

ఇది interviewer దాదాపు ఖచ్చితంగా అడిగే ప్రశ్న.

Singly linked list lo ప్రతి node కి `next` మాత్రమే ఉంటుంది. ఇప్పుడు ఒక node ని తీసేయాలంటే – **దాని ముందున్న node యొక్క `next` ని మార్చాలి**. కానీ singly list lo *ముందున్నది* ఎవరో తెలియదు. తెలుసుకోవాలంటే మొదటి నుంచి నడవాలి – **మళ్ళీ O(n)**.

	Singly	**Doubly**
ఒక node ని తీసేయడం (node చేతిలో ఉన్నప్పుడు)	**O(n)** – ముందున్నదాన్ని వెతకాలి	**O(1)** – `n.prev` ఉంది
Memory	node కి ఒక pointer	node కి రెండు pointers
మన అవసరం	✗	✓

ఒక ఆసక్తికరమైన ప్రత్యామ్నాయం: JavaScript `Map` చేర్చిన క్రమాన్ని గుర్తుంచుకుంటుంది, మరియు `delete` + `set` చేస్తే ఆ key చివరికి వెళ్తుంది. అంటే **కేవలం ఒక Map తో** LRU రాయొచ్చు – 15 పంక్తుల్లో.

కానీ interview lo **DLL version ఆశిస్తారు**, ఎందుకంటే అది భాష-స్వతంత్రం, మరియు అది మీకు ఆ structure తెలుసని చూపిస్తుంది. **ఉత్తమ ఎత్తుగడ:** DLL రాసి, ఆపై చెప్పండి – *"JavaScript lo Map insertion order ఇస్తుంది కాబట్టి ఇది ఒక Map తోనే వేయగలను, కానీ నేను స్పష్టమైన structure చూపించాలనుకున్నాను."* ఇది రెండూ తెలుసని చూపిస్తుంది.

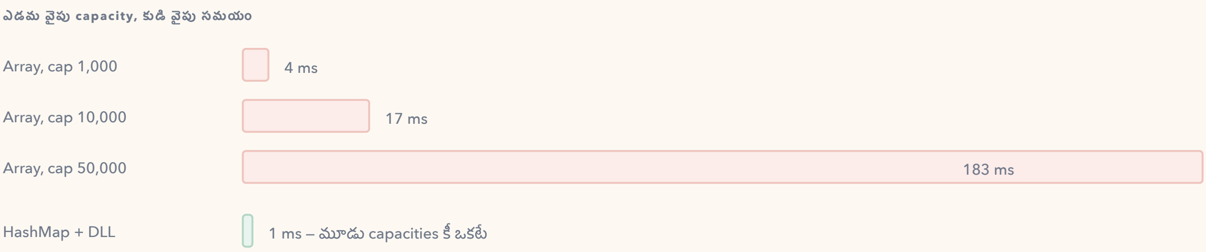
---

10. కొలత – 183 ms నుంచి 1 ms కి

అదే పరీక్ష, అదే machine:

```
capacity   1000 → 20,000 reads: 1 ms
capacity  10000 → 20,000 reads: 1 ms
capacity  50000 → 20,000 reads: 1 ms
```

అదే పని, రెండు STRUCTURES · 20,000 READS



ఇక్కడ ముఖ్యమైనది వేగం కాదు – వాలు

Array version యొక్క సమయం capacity తో పాటు పెరుగుతోంది. DLL version యొక్కది **పెరగట్లేదు**.
"183 ms నెమ్మది" అని కాదు – "ఇది 10 లక్షల entries lo చాలా సెకన్లు అవుతుంది" అని అర్థం.

Part 5 — LFU: పూర్తిగా వేరే ప్రశ్న

11. LRU vs LFU — ఏది ఎప్పుడు తప్పు జవాబు ఇస్తుంది

Interviewer అంటాడు: *"ఇప్పుడు LRU కాదు, **LFU** కావాలి."*

LFU = Least Frequently Used — అతి తక్కువసార్లు చదివినదాన్ని తీసేయడం. LRU **ఎప్పుడు** చదివారో చూస్తుంది; LFU **ఎన్నిసార్లు** చదివారో చూస్తుంది.

ఈ తేడా ఎప్పుడు ముఖ్యం? రెండూ తప్పు జవాబు ఇచ్చే సందర్భాలు ఉన్నాయి:

పరిస్థితి	LRU	LFU
ఒక పెద్ద scan (ఒక report అన్ని rows చదివింది)	✗ **విరిగిపోతుంది** — ఆ scan మొత్తం cache ని తుడిచేస్తుంది	✓ బతుకుతుంది — ఆ keys count 1 మాత్రమే
ఒక key ఇక పనికిరాదు (పాత promo code)	✓ నెమ్మదిగా జారిపోతుంది	✗ **ఇరుక్కుపోతుంది** — పాత పెద్ద count వల్ల శాశ్వతంగా ఉండిపోతుంది
కొత్త hot key	✓ వెంటనే పైకి వస్తుంది	✗ పాత keys తో పోటీ పడాలంటే చాలా సమయం

LFU యొక్క నిజమైన ఉచ్చు — దీన్ని INTERVIEW LO చెప్పండి

ఒక key రెండేళ్ళ క్రితం 10 లక్షల సార్లు చదవబడింది, ఆపై ఎవరూ ముట్టుకోలేదు. దాని count ఇప్పటికీ 10 లక్షలు. **అది ఎప్పటికీ తీసేయబడదు** — కొత్త, నిజంగా hot keys అన్నీ దాని చుట్టూ వచ్చిపోతూ ఉంటాయి.

దీన్ని **cache pollution** అంటారు, మరియు దీనికి నిజమైన పరిష్కారం **aging** — అప్పుడప్పుడూ అన్ని counts ని సగం చేయడం (ఉదా: ప్రతి 10,000 operations కి). ఇది **LFU-with-decay** లేదా **TinyLFU** అనే నమూనా.

"LFU రాయగలను" అనేది ఒక జవాబు. *"LFU రాయగలను, మరియు అది ఎక్కడ విరుగుతుందో, దాన్ని ఎలా సరిచేస్తారో తెలుసు"* — ఇది ఒక వేరే స్థాయి జవాబు.

12. మూడో విరుపు — naive LFU మరియు దాని O(n) scan

సహజమైన LFU: ప్రతి key కి ఒక count పెట్టి, తీసేయాల్సినప్పుడు **అతి తక్కువ count ఉన్నదాన్ని వెతకడం**.

```
class NaiveLfu {
  constructor(c) { this.capacity = c; this.vals = new Map(); this.counts = new Map();
}

  get(k) {
    if (!this.vals.has(k)) return -1;
```

```
    this.counts.set(k, this.counts.get(k) + 1);
    return this.vals.get(k);
  }

  put(k, v) {
    if (!this.vals.has(k) && this.vals.size >= this.capacity) {
      let victim = null, min = Infinity;
      for (const [key, cnt] of this.counts)        // ← ప్రతి eviction కీ అందరినీ చూడటం
        if (cnt < min) { min = cnt; victim = key; }
      this.vals.delete(victim); this.counts.delete(victim);
    }
    this.vals.set(k, v);
    this.counts.set(k, (this.counts.get(k) ?? 0) + 1);
  }
}
```

సరైన జవాబు ఇస్తోందా? చూద్దాం — `A` ని రెండుసార్లు, `B` ని ఒకసారి చదివి, `D` చేరుద్దాం:

```
counts: A=3 B=2 D=1
C పోయిందా (అతి తక్కువ)? true
```

సరైనదే. `C` ని ఎవరూ చదవలేదు కాబట్టి అదే పోయింది. కానీ ఖర్చు?

```
capacity   1000 → 2,000 evictions: 12 ms
capacity  10000 → 2,000 evictions: 69 ms
capacity  50000 → 2,000 evictions: 298 ms
```

మూడో విరుపు — §6 అదే కథ, కొత్త వేషంలో

మళ్ళీ అదే నమూనా: capacity 50 రెట్లు పెరిగితే, సమయం 25 రెట్లు పెరిగింది. **Eviction ఇప్పుడు O(n).**

మరియు ఇది **§6 కంటే ఘోరం**, ఎందుకంటే ఇక్కడ cache *నిండినప్పుడు* — అంటే అది ఎక్కువగా పనిచేస్తున్నప్పుడు — ప్రతి `put` కీ ఈ scan జరుగుతుంది.

పాఠం: "ఏది తీసేయాలి" అనే ప్రశ్నకి జవాబు కూడా O(1) కావాలి. LRU lo అది "జాబితా చివరిది" — ఉచితం. LFU lo అది అంత సులభం కాదు.

---

13. Step — Frequency buckets తో O(1) LFU

అసలు సమస్య: *"అతి తక్కువ count ఎంత?"* అని ప్రతిసారీ **వెతుకుతున్నాం**. బదులుగా దాన్ని **గుర్తుపెట్టుకోవచ్చు**.

ఆలోచన: **count వారీగా keys ని గుంపులుగా ఉంచడం**, మరియు ఇప్పటి **అతి తక్కువ count** ని ఒక variable lo ఉంచడం.

```
class LfuCache {
  #vals = new Map();                        // key → value
  #counts = new Map();                      // key → ఎన్నిసార్లు
  #buckets = new Map();                     // count → ఆ count ఉన్న keys (Set)
  #min = 0;                                 // ఇప్పటి అతి తక్కువ count

  constructor(capacity) { this.capacity = capacity; }

  #bucket(c) {
    if (!this.#buckets.has(c)) this.#buckets.set(c, new Set());
    return this.#buckets.get(c);
  }

  #bump(k) {                                // ఒక key యొక్క count ని ఒకటి పెంచడం
    const c = this.#counts.get(k);
    this.#bucket(c).delete(k);
    if (c === this.#min && this.#bucket(c).size === 0) this.#min = c + 1;   // ఖాళీ
అయింది
    this.#counts.set(k, c + 1);
    this.#bucket(c + 1).add(k);
  }

  get(k) { if (!this.#vals.has(k)) return -1; this.#bump(k); return
this.#vals.get(k); }

  put(k, v) {
    if (this.capacity === 0) return;
    if (this.#vals.has(k)) { this.#vals.set(k, v); this.#bump(k); return; }
    if (this.#vals.size >= this.capacity) {
      const victim = this.#bucket(this.#min).values().next().value;   // ← O(1)
      this.#bucket(this.#min).delete(victim);
      this.#vals.delete(victim); this.#counts.delete(victim);
    }
    this.#vals.set(k, v); this.#counts.set(k, 1);
    this.#bucket(1).add(k); this.#min = 1;                          // కొత్తది ఎప్పుడూ
count 1
  }
}
```

మూడు సూక్ష్మమైన, ముఖ్యమైన వివరాలు

1. `#min` ఎప్పుడు పెరుగుతుంది? ఒక key `c` నుంచి `c+1` కి వెళ్ళినప్పుడు, **మరియు** `c` bucket ఖాళీ అయినప్పుడు. అప్పుడు `#min = c + 1`. ఇది ఎందుకు సరైనది – ఎందుకంటే counts ఎప్పుడూ **ఒక్కొక్కటిగా** పెరుగుతాయి, కాబట్టి `#min` ఒక్క అడుగు మాత్రమే జరుగుతుంది, దూకదు.

2. కొత్త key వచ్చినప్పుడు `#min = 1`. ఇది స్పష్టం కానీ మర్చిపోతే బగ్: కొత్తదాని count 1, అంటే అదే ఇప్పుడు అతి తక్కువ.

3. Tie-break: `Set` వాడటం. రెండు keys కి ఒకే count ఉంటే ఎవరిని తీసేయాలి? JavaScript `Set` **చేర్చిన క్రమం** గుర్తుంచుకుంటుంది, కాబట్టి `values().next()` ఆ bucket lo **అతి పాతదాన్ని** ఇస్తుంది. అంటే మనకి ఉచితంగా **LFU-తో-LRU-tiebreak** వచ్చింది – ఇదే సరైన ప్రవర్తన.

దాన్ని పరీక్షిద్దాం:

```
const t = new LfuCache(2);
t.put('X', 1); t.put('Y', 2);      // రెండూ count 1
t.put('Z', 3);                      // X ముందు వచ్చింది → X పోవాలి
```

```
counts: A=3 B=2 C=1
D చేర్చాక: A=3 B=2 D=1 | C పోయిందా? true
tie-break: X పోయిందా? true | Y ఉందా? true
```

మరియు వేగం:

```
capacity   1000 → 2,000 evictions: 1 ms
capacity  10000 → 2,000 evictions: 1 ms
capacity  50000 → 2,000 evictions: 1 ms
```

298 ms → 1 ms, మరియు capacity తో పెరగట్లేదు.

Part 6 – ఒకే Cache, పలు policies

14. Step – Eviction ని బయట పెట్టడం

ఇప్పుడు మన దగ్గర రెండు పూర్తి caches ఉన్నాయి: `LruCache`, `LfuCache`. వాటిని పక్కపక్కన పెట్టి చూడండి – `get` **మరియు** `put` **యొక్క ఆకారం దాదాపు ఒకటే**:

```
విలువ ఉందా చూడు → లేకపోతే miss → ఉంటే "వాడాను" అని నమోదు చేసి విలువ ఇవ్వు
నిండిందా చూడు  → నిండితే ఒకరిని తీసేయి → కొత్తది చేర్చు
```

తేడా ఉన్నది ఒక్క చోట: **"వాడాను అని ఎలా నమోదు చేయాలి"** మరియు **"ఎవరిని తీసేయాలి"**. అంటే మారేది eviction నియమం, మారనిది cache యొక్క అస్థిపంజరం.

మారేదాన్ని బయట పెడదాం. ఒక policy కి కావలసినవి **నాలుగు** మాత్రమే:

```
class EvictionPolicy {
  onInsert(key) {}      // ఒక కొత్త key చేరింది
  onAccess(key) {}      // ఒక key చదవబడింది (లేదా తిరిగి రాయబడింది)
  onRemove(key) {}      // ఒక key పోయింది (eviction లేదా TTL వల్ల)
  victim() { throw new Error('subclass implement చేయాలి'); }   // ఎవరిని తీసేయాలి?
}
```

ఇప్పుడు మన రెండు versions ఈ ఒప్పందం లోపలికి వెళ్తాయి. **LRU policy** = మనం §8 lo రాసిన DLL, కానీ values లేకుండా – ఇది కేవలం **క్రమాన్ని** చూస్తుంది:

```
class LruPolicy extends EvictionPolicy {
  #nodes = new Map();
  #head = { key: null }; #tail = { key: null };
  constructor() { super(); this.#head.next = this.#tail; this.#tail.prev =
this.#head; }

  #unlink(n) { n.prev.next = n.next; n.next.prev = n.prev; }
  #front(n)  { n.next = this.#head.next; n.prev = this.#head;
               this.#head.next.prev = n; this.#head.next = n; }

  onInsert(key) { const n = { key }; this.#nodes.set(key, n); this.#front(n); }
  onAccess(key) { const n = this.#nodes.get(key); if (n) { this.#unlink(n);
this.#front(n); } }
  onRemove(key) { const n = this.#nodes.get(key); if (n) { this.#unlink(n);
this.#nodes.delete(key); } }
  victim()      { return this.#tail.prev.key; }
}
```

LFU policy = §13 యొక్క buckets, అదే ఒప్పందంలో:

```
class LfuPolicy extends EvictionPolicy {
  #counts = new Map(); #buckets = new Map(); #min = 0;
  #bucket(c) { if (!this.#buckets.has(c)) this.#buckets.set(c, new Set()); return
this.#buckets.get(c); }

  onInsert(key) { this.#counts.set(key, 1); this.#bucket(1).add(key); this.#min = 1;
}
  onAccess(key) {
    const c = this.#counts.get(key); if (c === undefined) return;
    this.#bucket(c).delete(key);
    if (c === this.#min && this.#bucket(c).size === 0) this.#min = c + 1;
    this.#counts.set(key, c + 1); this.#bucket(c + 1).add(key);
  }
  onRemove(key) {
    const c = this.#counts.get(key); if (c === undefined) return;
    this.#bucket(c).delete(key); this.#counts.delete(key);
  }
  victim() { return this.#bucket(this.#min).values().next().value; }
}
```

ఒక ముఖ్యమైన మార్పు గమనించండి: ఇప్పుడు policy lo **values లేవు** – కేవలం keys మరియు క్రమం. Values అన్నీ `Cache` lo ఉన్నాయి. ఈ విభజన వల్ల policy ని విడిగా test చేయొచ్చు, మరియు `Cache` కి eviction నియమం గురించి ఏమీ తెలియనవసరం లేదు.

ఇదే **Strategy pattern** – కానీ పేరు కంటే ముఖ్యమైనది ఇది: **కొత్త policy = ఒక కొత్త class, ఉన్న code lo సున్నా మార్పులు.**

---

15. Step – TTL, మరియు "గడియారం ఎవరిది"

TTL (time to live) = ఒక entry ఎంతకాలం చెల్లుబాటు అవుతుంది. Session cache, OTP, API token – వీటన్నిటికీ ఇది కావాలి.

అమలు సులభం: ప్రతి entry తో పాటు ఒక `expiresAt` ఉంచి, చదివేటప్పుడు తనిఖీ చేయడం.

కానీ ఇక్కడ ఒక **design ఉచ్చు** ఉంది, మరియు interviewers దీన్ని చూస్తారు:

```
#alive(e) { return e.expiresAt === null || Date.now() < e.expiresAt; }    // ✗
```

`Date.now()` ని class లోపల పిలిస్తే – **ఈ class ని test చేయడం దాదాపు అసాధ్యం**. "600 ms తర్వాత entry పోవాలి" అని test రాయాలంటే, ఆ test నిజంగా 600 ms **ఆగాలి**. పది అలాంటి tests ఉంటే మీ test suite ఆరు సెకన్లు నిద్రపోతుంది.

పరిష్కారం: **గడియారాన్ని బయట నుంచి ఇవ్వడం.**

```
constructor(capacity, { policy = new LruPolicy(), ttlMs = null,
                        clock = () => Date.now() } = {}) { … }
```

```
#alive(e) { return e.expiresAt === null || this.clock() < e.expiresAt; }   // ✓
```

Production lo default `Date.now` వాడుతుంది. Test lo మనం ఒక నకిలీ గడియారం ఇస్తాం:

```
let now = 1000;
const ttl = new Cache(10, { ttlMs: 500, clock: () => now });
ttl.put('session', 'abc');
console.log('వెంటనే:      ', ttl.get('session'));
now += 400;  console.log('400ms తర్వాత:', ttl.get('session'));
now += 200;  console.log('600ms తర్వాత:', ttl.get('session'));
```

```
వెంటనే:       abc
400ms తర్వాత: abc
600ms తర్వాత: undefined
```

ఒక్క millisecond కూడా ఆగకుండా TTL ని పూర్తిగా test చేశాం.

ఇది ఒక పెద్ద సూత్రం, ఇక్కడ చిన్నగా కనిపిస్తోంది

"సమయాన్ని inject చేయడం" – ఇది rate limiter, scheduler, session timeout, retry-with-backoff, token expiry… అన్నిటిలోనూ వస్తుంది. **Class లోపల `Date.now()` ఉంటే ఆ class ని test చేయలేరు.**

Deep Dive 06 (Elevator) lo ఇదే సూత్రం `step()` రూపంలో కనిపించింది. అక్కడ "ఒక tick = ఒక యూనిట్ సమయం", ఇక్కడ "clock ఒక function". **రెండూ ఒకే ఆలోచన.**

ఒక సూక్ష్మమైన ప్రశ్న: గడువు తీరిన entry – hit నా miss నా?

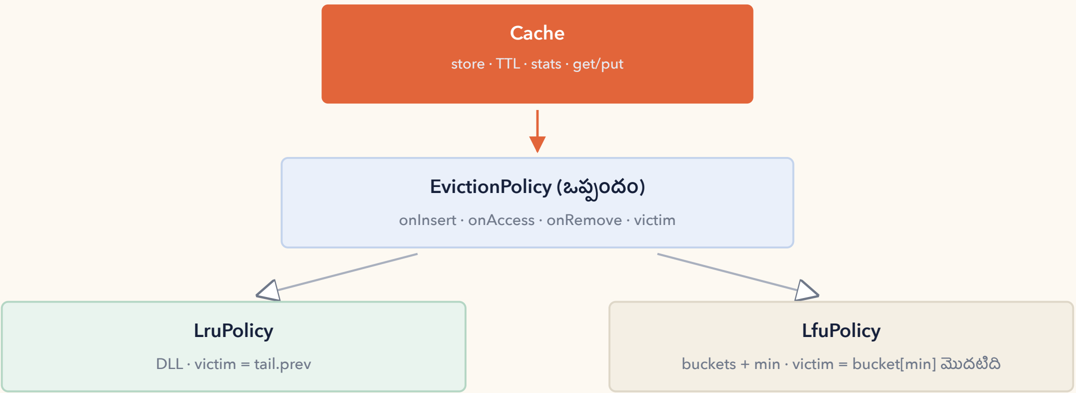
ఒక entry memory lo ఉంది, కానీ దాని TTL అయిపోయింది. `get` పిలిస్తే అది **miss** – ఎందుకంటే caller కి విలువ దొరకలేదు. మరియు దాన్ని అక్కడికక్కడే **తీసేయాలి** (lazy expiry), లేకపోతే అది memory lo కూర్చుని ఒక slot వృథా చేస్తుంది.

---

16. మొత్తం code ఒకే చోట · నడిపి చూద్దాం

నిర్మాణం · CACHE కి POLICY ఏమిటో తెలియదు

మూడు ముక్కలు, మరియు ఒక స్పష్టమైన సరిహద్దు




```
class Cache {
  #store = new Map();                      // key → { value, expiresAt }
  #hits = 0; #misses = 0; #evictions = 0;

  constructor(capacity, { policy = new LruPolicy(), ttlMs = null,
                          clock = () => Date.now() } = {}) {
    Object.assign(this, { capacity, policy, ttlMs, clock });
  }

  #alive(e) { return e.expiresAt === null || this.clock() < e.expiresAt; }
  #drop(key) { this.#store.delete(key); this.policy.onRemove(key); }

  get(key) {
    const e = this.#store.get(key);
    if (!e) { this.#misses++; return undefined; }
    if (!this.#alive(e)) { this.#drop(key); this.#misses++; return undefined; }
    this.policy.onAccess(key); this.#hits++;
    return e.value;
  }

  put(key, value, ttlMs = this.ttlMs) {
    if (this.capacity === 0) return;
    const expiresAt = ttlMs === null ? null : this.clock() + ttlMs;

    if (this.#store.has(key)) {            // ఉన్నదాన్ని మార్చడం — eviction అవసరం లేదు
      this.#store.set(key, { value, expiresAt });
      this.policy.onAccess(key);
      return;
    }
    if (this.#store.size >= this.capacity) {
      this.#drop(this.policy.victim());
      this.#evictions++;
    }
    this.#store.set(key, { value, expiresAt });
    this.policy.onInsert(key);
  }

  get size() { return this.#store.size; }
  get stats() {
```

```
    const total = this.#hits + this.#misses;
    return { hits: this.#hits, misses: this.#misses, evictions: this.#evictions,
             hitRate: total ? +(this.#hits / total).toFixed(2) : 0 };
  }
}
```

ఇప్పుడు **ఒకే `Cache` class, రెండు policies**:

```
console.log('— LRU —');
const lru = new Cache(3);
lru.put('A', 1); lru.put('B', 2); lru.put('C', 3);
lru.get('A');
lru.put('D', 4);
console.log('B పోయిందా?', lru.get('B') === undefined, '| A ఉందా?', lru.get('A') ===
1);

console.log('— అదే Cache, policy మాత్రమే మారింది —');
const lfu = new Cache(3, { policy: new LfuPolicy() });
lfu.put('A', 1); lfu.put('B', 2); lfu.put('C', 3);
lfu.get('A'); lfu.get('A'); lfu.get('B');
lfu.put('D', 4);
console.log('C పోయిందా?', lfu.get('C') === undefined, '| A ఉందా?', lfu.get('A') ===
1);
```

```
— LRU —
B పోయిందా? true | A ఉందా? true
— అదే Cache, policy మాత్రమే మారింది —
C పోయిందా? true | A ఉందా? true
```

రెండు సందర్భాల్లోనూ `Cache` class ఒక్క అక్షరం కూడా మారలేదు. మారినది ఒక constructor argument మాత్రమే.

దశల నుంచి ఇక్కడికి – ఏమి చేరింది

ఎక్కడ నుంచి	ఏమి చేరింది	ఎందుకు
§3	`capacity`, `Map`	మౌలిక అస్థిపంజరం
§4 (విరుపు)	వాడకపు క్రమం	FIFO వేడి key ని తీసేసింది
§6 (విరుపు)	DLL, sentinels	Array touch O(n) – 183 ms
§12 (విరుపు)	Frequency buckets, `#min`	LFU eviction scan O(n) – 298 ms
§14	`EvictionPolicy` ఒప్పందం	రెండు caches యొక్క ఆకారం ఒకటే
§15	`expiresAt`, `clock`	TTL, మరియు test చేయగలగడం
ఇక్కడ	`stats`	Cache పనిచేస్తోందో లేదో కొలవడానికి

Stats ఎందుకు

ఒక cache **పనిచేస్తోందో లేదో** ఎలా తెలుసు? Hit rate చూడాలి.

```
{ hits: 2, misses: 1, evictions: 0, hitRate: 0.67 }
```

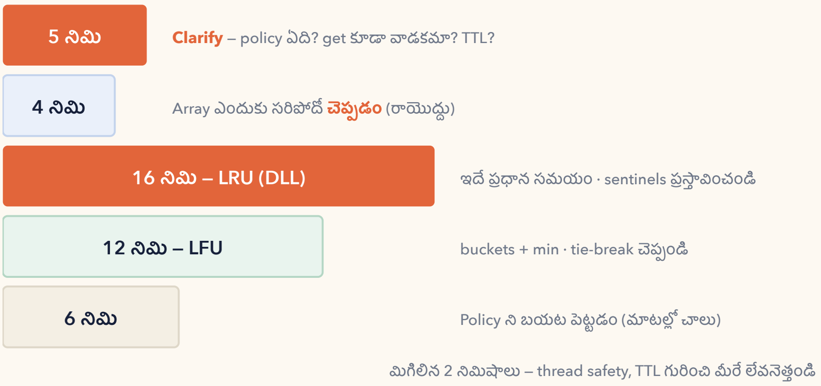
> **Interview lo ఇది ఒక మంచి అదనపు మాట:** *"I'd expose hit rate, because a cache with a 5% hit rate is worse than no cache – you're paying the memory and the lookup and getting nothing. Without the metric you'd never know."*

Part 7 – Interview lo

17. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి

45 నిమిషాల TIME BUDGET

ఈ problem lo code ఎక్కువ, మాటలు తక్కువ – కానీ మొదటి 5 నిమిషాలు మాటలే



వరుస ముఖ్యం

LRU ని పూర్తిగా ముగించండి, ఆపై LFU. సగం LRU + సగం LFU = రెండూ అసంపూర్ణం. ఒకటి పూర్తిగా పనిచేస్తే, రెండోదాన్ని మాటల్లో వివరించినా సరిపోతుంది.

ఏమి తప్పక చెప్పాలి

1. **Array ఎందుకు సరిపోదు** – `indexOf` + `splice` రెండూ O(n) (§7). దీన్ని **రాయకుండానే** చెప్పండి, సమయం ఆదా.
2. **Sentinel nodes ఎందుకు** (§8) – null checks తొలగిపోతాయి.
3. **Doubly ఎందుకు** (§9) – singly lo `prev` లేదు కాబట్టి delete O(n).
4. **LFU tie-break** (§13) – ఒకే count ఉన్న రెండు keys మధ్య LRU.
5. **LFU యొక్క పాత-hot-key సమస్య** (§11) – ఇది చెప్తే మీరు నిజంగా ఆలోచించారని తెలుస్తుంది.

18. నోటితో చెప్పాల్సిన English script

"A couple of questions first. Which eviction policy — LRU? Does a `get` count as a use, or only writes? And do I need TTL, or is capacity the only limit?

I'll do LRU first. The obvious approach is a map plus an array holding keys in use order, and moving a key to the end when it's read. That's correct but it's O(n) per read — `indexOf` scans, and `splice` shifts everything after it. On a cache of fifty thousand entries that's measurably slow, which defeats the point of the cache.

So I need two things at once: find the node in O(1), and move it in O(1). A hash map gives me the first, a doubly linked list gives me the second. Map from key to node; the node lives in the list; most-recent at the front, so the victim is always `tail.prev`.

It has to be doubly linked — with a singly linked list, removing a node means finding its predecessor, and that's a walk. I'll also use sentinel head and tail nodes so no node is ever at a boundary; that removes every null check from remove and insert, which is where these bugs usually live.

For LFU the answer changes shape. The naive version scans every key to find the minimum count on each eviction, which is O(n) exactly when the cache is busiest. Instead I keep a bucket per count — count to a set of keys — and track the current minimum. Incrementing moves a key one bucket up, and the minimum only ever advances by one, so I can maintain it in O(1). For the victim I take the first key out of the minimum bucket, and because a JavaScript Set preserves insertion order, that gives me LRU tie-breaking within a frequency for free.

One real problem with LFU: a key that was hot two years ago keeps a huge count and never gets evicted, so it pollutes the cache forever. Production systems fix that with aging — periodically halving all counts. That's roughly what TinyLFU does.

Both of these are the same cache with a different eviction rule, so I'd put eviction behind a small interface — `onInsert`, `onAccess`, `onRemove`, `victim` — and a new policy becomes a new class rather than an edit.

For TTL, I'd take the clock as a constructor parameter rather than calling `Date.now()` inside, so expiry can be tested without any test actually sleeping."

---

19. Follow-ups — thread safety, distributed cache

Follow-up	జవాబు	మారే classes
"Random eviction (test కోసం)"	కొత్త `RandomPolicy`	**+1 కొత్తది, 0 edits**

Follow-up	జవాబు	మారే classes
"ప్రతి entry కి వేరే TTL"	`put(k, v, ttlMs)` ఇప్పటికే ఉంది	**0**
"Eviction అయినప్పుడు callback"	Constructor lo ఒక `onEvict` hook	**+1 parameter**
"Cache stats"	`stats` ఇప్పటికే ఉంది	**0**
"LFU lo aging"	`LfuPolicy` lo ప్రతి N operations కి counts సగం	Policy మాత్రమే
"Thread-safe కావాలి"	కింద చూడండి	–
"పలు servers (Redis)"	కింద చూడండి	–

Thread safety

> *"JavaScript is single-threaded, so this is already safe here. In Java it wouldn't be – `get` isn't read-only, it mutates the list, so two concurrent reads can corrupt it. The simple answer is one lock around get and put. The problem is that makes every read serialise, which for a cache is exactly the wrong trade-off, because reads are the common case.*
>
> *What production caches do instead – Caffeine is the usual example – is buffer the access records. The read returns immediately and just appends the access to a ring buffer, and the ordering is applied later in a batch. So reads stay fast and the eviction order becomes slightly approximate. That's a deliberate trade: you give up exact LRU ordering for concurrency, and for a cache that's almost always worth it."*

ఈ జవాబు చాలా బలమైనది ఎందుకంటే ఇది ఒక **నిజమైన trade-off** చెప్తోంది – "lock పెడతాను" అనే సాధారణ జవాబు కంటే ఒక స్థాయి పైన.

Distributed cache

> *"అదే interface, వేరే implementation. `get` / `put` Redis కి వెళ్తాయి. కానీ అప్పుడు కొత్త సమస్యలు: **cache stampede** (ఒక hot key expire అయినప్పుడు వెయ్యి requests ఒకేసారి DB కి వెళ్ళడం – దీనికి single-flight లేదా probabilistic early refresh), మరియు **invalidation** (ఒక server data మార్చితే మిగతావాళ్ళకి ఎలా తెలుస్తుంది – pub/sub లేదా చిన్న TTL). ఈ రెండూ LRU/LFU కంటే కష్టమైన సమస్యలు."*

---

20. ఏమి నేర్చుకున్నాం

ఆలోచన	ఇక్కడ ఎలా కనిపించింది	ఇంకెక్కడ వస్తుంది
సరైన జవాబు సరిపోదు; ఖర్చు కూడా చూడాలి	Array version సరైనది, కానీ O(n) (§6)	ప్రతి data-structure ఎంపికలో

ఆలోచన	ఇక్కడ ఎలా కనిపించింది	ఇంకెక్కడ వస్తుంది
రెండు structures కలపడం	HashMap (వెతకడం) + DLL (కదలిక)	LFU buckets, LinkedHashMap, indexes
"కనిష్ఠం" ని వెతకొద్దు, గుర్తుపెట్టుకో	`#min` (§13)	Min-heap, running max, sliding window
మారే నియమాన్ని బయట పెట్టడం	`EvictionPolicy` (§14)	Pricing, allocation, retry, rate limit
సమయాన్ని inject చేయడం	`clock` (§15)	Rate limiter, scheduler, session, token
ఖచ్చితత్వాన్ని వేగం కోసం వదులుకోవడం	Caffeine యొక్క buffered reads (§19)	Approximate counters, sampling, sketches

ఇక్కడి నుంచి ఎక్కడికి

ఈ series lo తర్వాతివి: **03 Rate Limiter** (అక్కడ మళ్ళీ `clock` injection, మరియు sliding window), **04 BookMyShow** (seat locking = Deep Dive 01 యొక్క race, పెద్ద రూపంలో), **05 Splitwise**.

ఇప్పటికే వచ్చినవి: **01 Parking Lot** · **06 Elevator**.

వేగవంతమైన revision కోసం – `LLD_Design_Problems_Telugu.pdf` lo Problem 07. సాధారణ సూత్రాల కోసం – `LLD_Telugu.pdf` §29 (Strategy), §36 (Concurrency). Data structures కోసం – `DSA_00_Foundations_Telugu.pdf`.

LRU & LFU Cache – అడుగు అడుగునా · ఈ doc lo ఉన్న ప్రతి output, ప్రతి timing నిజంగా `node` lo run చేసి తీసినదే ✅